

Table des matières

1. Pour commencer
2. Positionnement d'un gestionnaire ; alarmes et pauses
3. Prise en main des ensembles de signaux
4. Les signaux temps-réels

1. Pour commencer

Le but de ce TP est de manipuler les appels système relatifs à la gestion des signaux.

Pour l'ensemble des questions suivantes, vous devez utiliser les fonctions C qui ont été introduites dans les cours correspondants. À noter que la gestion d'erreurs doit être correctement mise en place pour tous les appels systèmes et l'utilisation d'un makefile est obligatoire.

2. Positionnement d'un gestionnaire ; alarmes et pauses

Dans cet exercice, nous allons vérifier le comportement de la fonction `sigaction`.

1. Écrivez un programme qui se met en pause pendant 10 secondes à l'aide de la fonction `alarm`. Une fois les 10 secondes passées, le programme affiche un message puis termine "normalement".
2. Écrivez un programme qui se met en pause pendant 10 secondes et qui compte le nombre de signaux SIGINT reçus. Une fois la pause terminée, le programme affiche le nombre de signaux reçus.

Lorsqu'un gestionnaire est appelé à la réception d'un signal, faites en sorte que votre programme reprenne le cours normal d'exécution (dans le `main`, par exemple) avant de s'arrêter.

3. Prise en main des ensembles de signaux

L'objectif de cet exercice est de tester le fonctionnement de `sigprocmask`, `sigsuspend` et `sigpending`, ainsi que les fonctions de gestion des ensembles de signaux.

Nous souhaitons écrire deux programmes : un *serveur* et un *client*. Le *serveur* affiche son PID puis il se met en attente du signal SIGINT à l'aide de `sigsuspend`. Une fois le signal reçu, il indique s'il a reçu ou non le signal SIGUSR1 pendant son attente. Le *client* prend en argument le PID du serveur (il doit vérifier ses arguments...). Il commence par vérifier que le PID est valide puis il envoie ensuite 3 signaux SIGUSR1 avec une pause de 1 seconde entre chaque. Puis il envoie le signal SIGINT.

1. Le *client* :

- a) Comment vérifier la présence d'un processus à l'aide d'un signal ?
- b) Écrivez le *client*.

2. Le *serveur* :

- a) Pourquoi doit-on bloquer le signal SIGUSR1 ?
- b) Pourquoi est-ce nécessaire de spécifier un gestionnaire pour SIGINT ?
- c) Écrivez le *serveur*.

3. Vérifiez le fonctionnement de vos programmes. Vérifiez leur comportement en supprimant le positionnement du gestionnaire, le blocage des signaux, etc.

4. Les signaux temps-réels

Pour illustrer le fonctionnement des signaux temps réel et l'échange de données, nous allons écrire deux applications client/serveur qui s'échangent des données via les signaux temps réels. Le but du client est d'arrêter le serveur en utilisant une bonne combinaison de signaux. Au début, le serveur se met en attente d'un signal SIGRTMIN. Dès qu'il le reçoit, il génère un nombre aléatoire X dans [0; 3] puis l'envoie au client à l'aide d'un signal SIGRTMIN. Il attend ensuite de recevoir un signal SIGRTMIN + X. S'il ne reçoit pas le bon signal, il recommence au début. En cas de réussite, il réitère l'opération 2 fois de suite puis s'arrête.

Exemple d'exécution :

- Le client envoie un signal SIGRTMIN
- Le serveur répond avec X = 2
- Le client envoie un signal SIGRTMIN+2
- Le serveur répond avec X = 1
- Le client envoie un signal SIGRTMIN+1
- Le serveur s'arrête (et le client aussi...)

1. Comment le serveur peut-il connaître le PID du client ?

2. Commencez par écrire un serveur et un client simples : le client envoie SIGRTMIN et le serveur répond par SIGRTMIN. Utilisez `sigwaitinfo` pour l'attente du signal et `sigqueue` pour l'envoi. N'oubliez pas de supprimer SIGINT lors du blocage des signaux : c'est toujours utile d'arrêter le client ou le serveur rapidement...

3. Complétez les applications.